

Understanding AMIRIS

A Complete Reference

Inputs, outputs, folders, every file, what each one contains, what it means, and how the pieces fit together - written in plain language throughout.

Prepared 20 July 2026

1. What Is AMIRIS?

AMIRIS (Agent-based Market model for the Investigation of Renewable and Integrated energy Systems) is a free, open-source computer program, built by the German Aerospace Center (DLR), that simulates the German electricity market. It works out what the wholesale electricity price would be for every hour of a year, by recreating - in software - the actual buying and selling process that sets that price in real life.

The core idea, in one analogy:

Think of AMIRIS as a detailed simulation game. Instead of one formula guessing a price, it creates many small virtual characters - called 'agents' - each standing in for a real type of market participant: one for all the nuclear plants, one for wind turbines, one for the people buying electricity, and so on. Every simulated hour, each character does what its real-world counterpart would do, and a real price emerges from their collective behaviour - the same way real prices emerge from real buyers and sellers.

Why this matters for the current project

A department currently pays a private company (Energy Brainpool) for exactly this kind of hourly price data. AMIRIS is a free, transparent alternative - if it can be shown to produce similarly trustworthy results, it could replace that purchased data going forward. Everything in this document explains what AMIRIS needs to be given, and what it hands back, so that comparison can be made fairly.

2. The Big Picture: How Data Flows Through AMIRIS

Every AMIRIS run follows the same five-step journey:

- Step 1 - Real facts go in: historical or assumed data about demand, weather-driven renewable output, power plant fleets, and fuel prices are collected into a set of input files.
- Step 2 - A 'recipe' describes the players: a scenario configuration describes every virtual agent (each plant type, each trader, the market itself) and the rules for how they interact.
- Step 3 - The engine runs the year: a Java program steps through every hour of the year, having every agent bid into a simulated auction, and works out that hour's market-clearing price.
- Step 4 - Results are extracted: the raw output is converted into readable spreadsheets (CSV files), one per agent type.
- Step 5 - People check the results: the calculated prices (and other outputs) are read, plotted, and compared against reality or against other data sources.

The one thing to hold onto throughout this document:

AMIRIS does not invent or guess its inputs - those are handed to it as known facts. Its only job is to calculate what a realistic market, full of realistic buyers and sellers, would do with those facts. Everything below is either something that goes IN (a fact AMIRIS is given) or something that comes OUT (a result AMIRIS calculates).

3. The Folder Structure

Folder / File	What it is
amiris-core...jar	The simulation engine itself - a runnable Java program. This is the 'brain' that actually does the calculation.
py313env\	The Python toolbox (amirispy, fameio) used to prepare inputs, launch the engine, and turn its raw output into readable files.
examples\backtest\Germany<year>\	A self-contained 'recipe' folder for one specific historical year - everything AMIRIS needs to reproduce that year's market lives here.
result\ (or result_Germany<year>)\	Where the answers land after a run finishes.

Inside every scenario 'recipe' folder, the layout is always the same:

File / Folder	What it is
scenario.yaml	The master recipe card: which year, exactly when it starts/ends, and which other files to pull in.
schema.yaml	A large, machine-generated technical rulebook the software uses internally. Not something a person edits by hand.
agents\	Description cards for every virtual participant - how big is the nuclear fleet, how much wind and solar capacity exists, how big is the battery storage, etc.
contracts\	The rulebook for who is allowed to trade with whom (e.g. power plants sell through a trader, the trader sells into the market).
timeseries\	The real, historical hour-by-hour input data for that year - see Section 4 for every file explained individually.

4. Every Input File, Explained (the timeseries\ folder)

These files are the real, measured 'ingredients' for a given year. None of them are produced by AMIRIS - they are handed to it. Every file uses the same simple format: no header row, one line per hour, written as 'date_time;value'.

Demand

File	What it contains, in plain terms
load.csv	How much electricity all of Germany actually used, for every hour of the year - like a nationwide electricity meter reading.

Renewable output (how much sun/wind/water was actually available)

File	What it contains, in plain terms
solar_profile.csv	Fraction (0 to 1) of maximum possible solar output achieved each hour - 0 at night, up to the observed peak on a sunny midday.

File	What it contains, in plain terms
wind_onshore_profile.csv / wind_offshore_profile.csv	Same idea for onshore and offshore wind turbines - driven by how windy it actually was.
run_of_river_profile.csv	Same idea for river-flow hydro power - steadier than wind/solar, since river flow changes slowly.
biomass_profile.csv	How biogas/biomass plants were actually dispatched - these are more controllable than weather-driven sources.
other_res_profile.csv	Same idea for smaller, miscellaneous renewable sources grouped together.

Fuel and carbon prices

File	What it contains, in plain terms
co2_price.csv	What it cost, per tonne, to be allowed to emit CO2 (the EU's carbon trading scheme price).
hard_coal_price.csv / natural_gas_price.csv / oil_price.csv	What it cost power plants to buy each fuel, delivered, per unit of energy content. These change slowly, so the file only has a data point every week or two, not every hour.

Power plant availability

File	What it contains, in plain terms
nuclear_outage.csv / lignite_outage.csv / hard_coal_outage.csv / natural_gas_outage.csv	What fraction of that fleet was offline for maintenance or breakdowns, hour by hour - power plants are not available 100% of the time, the same way a car needs servicing.
*_must_run.csv (where present)	The opposite idea: the minimum amount of that fleet that HAD to keep running, even if uneconomical (e.g. for grid stability or heat supply to a town).
backup_storage_dispatch.csv (2017 only)	A special file added specifically to capture an unusual real event - the January 2017 'Kalte Dunkelflaute' (a multi-day cold, calm spell with almost no wind or solar output).

5. The Configuration Files (the agents\ and contracts\ folders)

These files do not contain hour-by-hour data - they describe the 'cast of characters' and the rules of the game, once per scenario.

File	What it describes, in plain terms
agents\Conventionals.yaml	One entry per fuel type (nuclear, lignite, hard coal, gas, oil): how much capacity exists, how efficient the plants are, how much it costs to run them, how much CO2 they emit, and how aggressively they're allowed to price their offers (a 'markup' range).

File	What it describes, in plain terms
agents\RenewablesAndPolicy.yaml	One entry per renewable technology and subsidy scheme: how much capacity exists, which weather profile file it uses, and what support payment it receives.
agents\Demand.yaml	Describes the agent responsible for buying enough electricity to cover the load.csv figures each hour.
agents\Storage.yaml	Describes battery/storage operators: how much they can hold, how fast they can charge/discharge, and how many separate competing operators exist.
agents\MarketsAndForecast.yaml	Describes the marketplace itself, and any internal forecasting agents used by traders to plan ahead.
contracts*.yaml	Defines who is allowed to sell to whom - e.g. a power plant sells through its trader, the trader sells into the market. Without these, the agents described above would have no path to actually trade.

6. Every Output File, Explained (the result\ folder)

After a run finishes, these are the files produced. Every CSV here is one row per hour per agent, and can be traced back to a specific virtual participant by its AgentId.

File	What it contains, in plain terms
output.pb	The master answer file, in a compact computer-only format - every CSV below is extracted from this one file.
metadata.json	An instruction manual describing exactly what every column in every CSV means - the authoritative reference if a column name is unclear.
DayAheadMarketSingleZone.csv	THE key result: the electricity price AMIRIS calculated for every hour of the year, plus how much energy was traded and the overall system cost. This is the file compared against real or purchased price data.
ConventionalPlantOperator.csv	What the coal/gas/nuclear/oil plants did each hour: how much they produced, how much CO2 they emitted, how much fuel they burned, and how much money they made.
ConventionalTrader.csv	The trading side for conventional plants: how much energy was offered, requested, and actually awarded at auction.
RenewableTrader.csv / NoSupportTrader.csv / SystemOperatorTrader.csv	What the various categories of wind/solar/biomass sellers did each hour, including any subsidy payments received - split into three files because German renewables are marketed under different support arrangements.
VariableRenewableOperator.csv / Biogas.csv	The plant-operator-level detail behind the renewable traders above: production, revenue, and costs per technology.
GenericFlexibilityTrader.csv	What battery/storage operators did: when they charged, when they discharged, at what price, and how full they were left at the end of each hour.
DemandTrader.csv	How much electricity buyers requested and were actually given each hour.
SensitivityForecaster.csv	Internal price/volume forecasts some agents use to plan ahead - reflects imperfect foresight, not the final market outcome.

7. How the Pieces Interact: the Mechanism

Every simulated hour, the following happens, independently, 8,760 times over the course of a year:

- Every seller (a power plant or renewable operator) works out its cost and submits an offer to sell a certain amount of electricity at a certain price.
- Every buyer submits how much electricity it needs.
- The market lines up all the sell offers from cheapest to most expensive - this ordering is called the 'merit order' - and accepts them one by one until enough electricity is queued up to cover demand.
- The price of the last, most expensive offer accepted becomes that hour's price, and everyone whose offer was accepted is paid that same price - this is exactly how the real German day-ahead auction works.

Why renewables usually 'win' first:

Wind and solar have no fuel cost, so they can afford to offer their electricity very cheaply (often at or near zero) and still be accepted first in the queue. This is why more renewable capacity tends to push prices down - it's the same mechanism, not a separate rule.

8. Assumptions Built Into How AMIRIS Behaves

Beyond the raw data, AMIRIS's behaviour depends on a handful of built-in assumptions - worth knowing, since these are exactly the kind of thing that can explain a gap between AMIRIS's result and a real or purchased price.

- Bidding markups: each fuel type is allowed to price its offer within a min/max range around its production cost, rather than at one fixed number - nuclear and lignite are even allowed to bid below zero, since shutting down and restarting can cost more than running at a loss for a few hours.
- No cross-border trade (in the scenarios used so far): Germany is treated as one closed market, with no electricity imported from or exported to neighbouring countries, unless a scenario is explicitly built to include it.
- Perfect knowledge of the past, not the future: AMIRIS is handed real historical facts for the year being simulated - it is not forecasting or guessing them.
- Simple, rule-based agents: each agent follows a straightforward bidding strategy (cost plus a markup), not a fully game-theoretic optimal strategy against every other agent.
- A fixed, non-daylight-saving hourly clock: AMIRIS's internal calendar runs as a continuous 8,760-hour grid and does not shift for daylight saving time the way real clocks do - worth knowing when lining its output up against real-world timestamped data.